OrderSystem

A cloud-native order platform shipped through a full DevSecOps pipeline: CI with security gates, GitOps delivery, Kubernetes, Terraform on AWS and live observability



What I built

2

Spring Boot microservices

order-service + inventory-service on Java 25

7

CI/CD pipeline jobs

build, scan, package, publish and deploy

3

security scanners

Gitleaks, Trivy FS, Trivy image

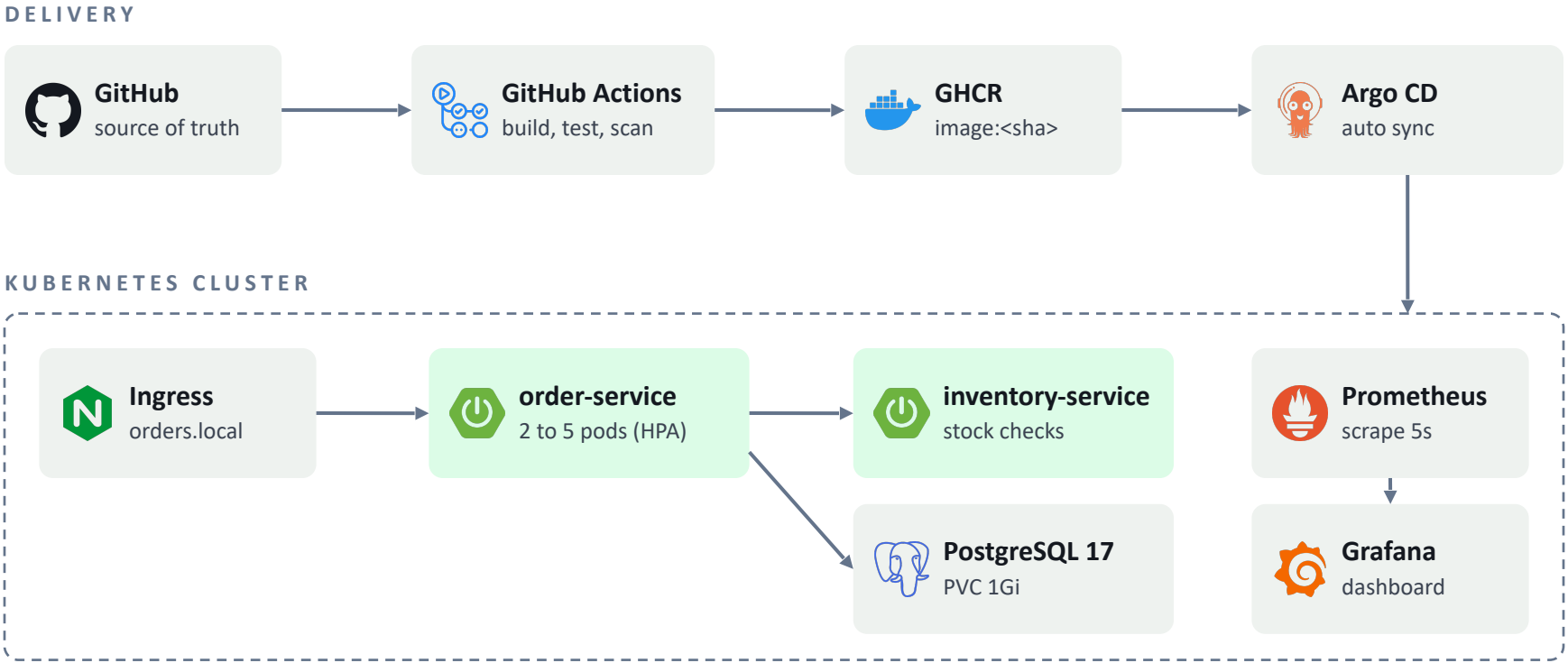
2

AWS availability zones

VPC, EKS and RDS via Terraform

Goal: every change goes from git push to running pods with no manual steps, and is tested, scanned and observable on the way.

Architecture overview



Application layer

order-service :8080

- POST, GET /api/orders and /api/orders/{id}
- Checks stock with inventory-service before saving
- Spring Data JPA on PostgreSQL 17
- Actuator health probes and metrics

inventory-service :8081

- GET /api/inventory/{productCode}
- In-memory stock catalogue
- Same health and metrics endpoints
- Reached over cluster DNS

Multi-stage Docker builds: JDK stage compiles, JRE stage ships only the jar. Local dev runs on Docker Compose.

CI/CD pipeline on GitHub Actions

1. Build & Test (Postgres service)
2. Gitleaks secret scan
3. Trivy filesystem scan
4. Helm lint + template
5. Terraform fmt + validate

6 Build & scan image

- Matrix over both services
- Trivy image scan
- Push ghcr.io/…:<sha>

7 Update image tags

- Rewrites tags in kubernetes/ and Helm values
- Commits to main

Runs on every push and PR to main. Jobs 6 and 7 publish only from main, after all checks pass.

Security built into the pipeline

Secret scanning

Gitleaks scans the full git history and blocks the pipeline on a finding

Dependency + IaC scan

Trivy checks Maven dependencies, K8s and Terraform misconfiguration

Image scanning

Trivy reports HIGH and CRITICAL CVEs that have a fix in each image

Supply chain

Every third-party action is pinned to a commit SHA

Least privilege

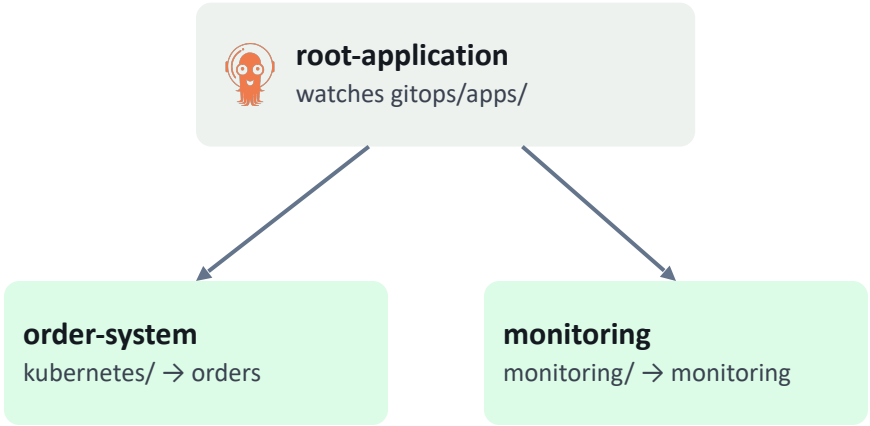
Read-only token by default; jobs request only the scopes they need

Secrets handling

DB password via TF_VAR, K8s Secret for creds, tfvars gitignored

GitOps with Argo CD

1. git push to main
2. CI pushes image:<sha> to GHCR
3. CI commits the new tag to kubernetes/
4. Argo CD detects the commit and syncs



prune: removed from Git, removed from cluster
selfHeal: manual kubectl changes are reverted

Kubernetes resources

Deployments

order-service x2, inventory-service x1, Postgres, all with requests and limits

Probes

Readiness gates traffic, liveness restarts stuck containers

Autoscaling

HPA scales order-service from 2 to 5 pods at 70% CPU

Ingress

NGINX routes orders.local to the order-service Service

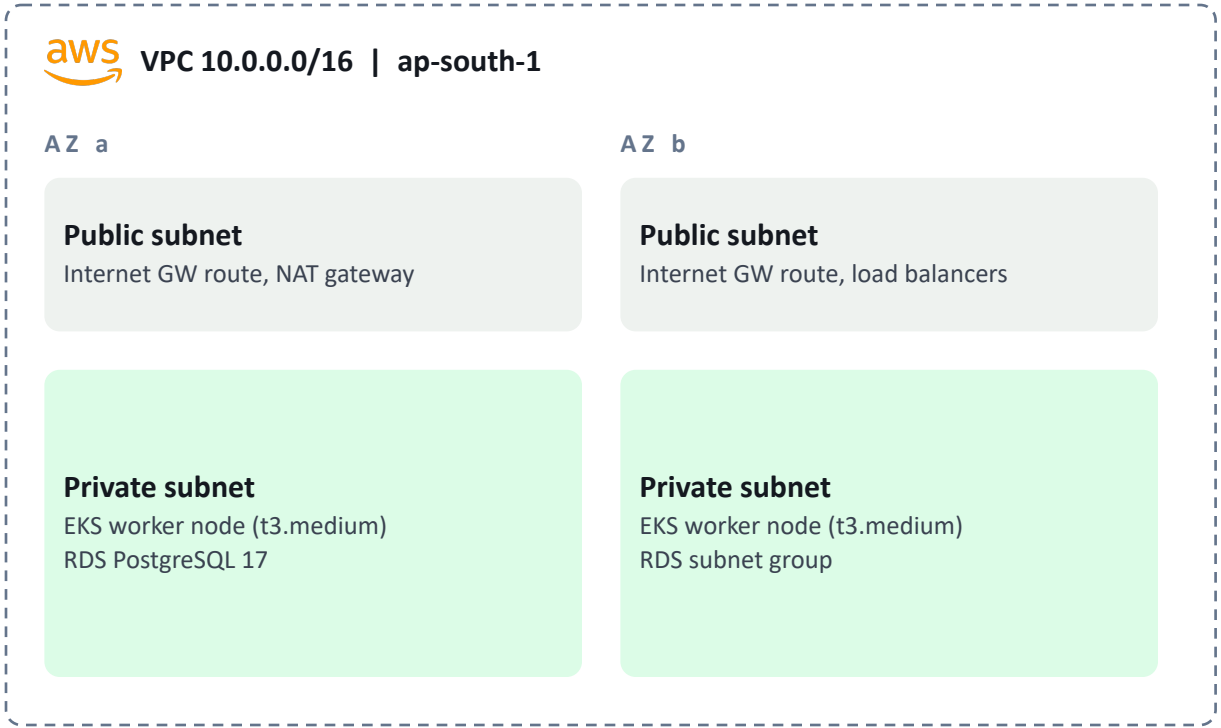
Config + Secrets

ConfigMap for the DB URL, Secret for credentials, injected as env

Storage

1Gi PersistentVolumeClaim keeps Postgres data across restarts

AWS infrastructure with Terraform



Workflow

- terraform init
- terraform plan
- terraform apply
- update-kubeconfig
- terraform destroy

EKS control plane managed by AWS; nodes only in private subnets.

Observability



Prometheus targets

order-service **UP**
inventory-service **UP**
prometheus **UP**

Scrapes /actuator/prometheus every 5 seconds



Alert rules

OrderServiceDown
target down for 30s
InventoryServiceDown
target down for 30s
HighOrderErrorRate
sustained HTTP 5xx



Grafana dashboard

- Service up status
- JVM heap usage
- Request rate by endpoint and status
- 5xx error count

Datasource and dashboard provisioned from Git

Problems hit and how I fixed them

Tests failed in CI

Cause: The context test needs a database

Fix: Added a postgres:17 service container to the job

EKS nodes cannot join

Cause: Private subnets had no route out

Fix: Added a NAT gateway and private route table

Argo CD could not sync

Cause: kind cluster config sat inside the synced folder

Fix: Moved it to cluster/ and built the app of apps folder

Pods stuck in a restart loop

Cause: Liveness probe killed the slow JVM boot

Fix: Startup probe and a bigger CPU request

Live walkthrough

github.com/parthdagia05/devOps-final


Codebase


CI/CD run


Terraform + AWS


Argo CD sync


kubectl status


Prometheus targets


Grafana dashboard